

NTUplace4h: A Novel Routability-Driven Placement Algorithm for Hierarchical Mixed-Size Circuit Designs

Meng-Kai Hsu, Yi-Fang Chen, Chau-Chin Huang, Sheng Chou, Tzu-Hen Lin,
Tung-Chieh Chen, and Yao-Wen Chang, *Fellow, IEEE*

Abstract—A wirelength-driven placer without considering routability could introduce irresolvable routing-congested placements. Therefore, it is desirable to develop an effective routability-driven placer for modern mixed-size designs employing hierarchical methodologies for faster turnaround time. In this paper, we propose a novel routability-driven analytical placement algorithm for hierarchical mixed-size circuit designs. This paper presents a novel design hierarchy identification technique to effectively identify design hierarchies and guide placement for better wirelength and routability. The proposed algorithm optimizes routability from four major aspects: 1) narrow channel handling; 2) pin density; 3) routing overflow optimization; and 4) net congestion optimization. Routability-driven legalization and detailed placement are also proposed to further optimize routing congestion. Compared with the participating teams for the 2012 ICCAD Design Hierarchy Aware Routability-driven Placement Contest, our placer can achieve the best quality (both the average overflow and wirelength) and the best overall score (by additionally considering running time).

Index Terms—Global routing, layouts, physical design, placement.

Manuscript received January 31, 2014; revised May 30, 2014; accepted August 13, 2014. Date of current version November 18, 2014. This work was supported in part by Genesys Logic, IBM MediaTek, SpringSoft, Synopsys, TSMC, Academia Sinica, and the NSC of Taiwan under Grant NSC 103-2918-I-002-012, Grant NSC 102-2221-E-002-235-MY3, Grant NSC 102-2923-E-002-006-MY3, Grant NSC 101-2221-E-002-191-MY3, Grant NSC 100-2221-E-002-088-MY3, Grant NSC 99-2221-E-002-207-MY3, Grant NSC 99-2221-E-002-210-MY3, Grant NSC 98-2221-E-002-119-MY3, and Grant NSC 97-2221-E-002-237-MY3. Preliminary versions of this paper were presented at the 2013 ACM/IEEE Design Automation Conference (DAC'13) in June 2013 [1] and 2011 IEEE/ACM International Conference on Computer-Aided Design (ICCAD'11) in November 2011 [2]. This paper was recommended by Associate Editor C. C.-N. Chu.

M.-K. Hsu, Y.-F. Chen, C.-C. Huang, S. Chou, and T.-H. Lin are with the Graduate Institute of Electronics Engineering, National Taiwan University, Taipei 106, Taiwan (e-mail: kaie@eda.ee.ntu.edu.tw; yifang@eda.ee.ntu.edu.tw; wlkb83@eda.ee.ntu.edu.tw; chousheng@eda.ee.ntu.edu.tw; kenylin@eda.ee.ntu.edu.tw).

T.-C. Chen is with Synopsys, Inc., Hsinchu 300, Taiwan (e-mail: donnie.chen@synopsys.com).

Y.-W. Chang is with the Department of Electrical Engineering and Graduate Institute of Electronics Engineering, National Taiwan University, Taipei 106, Taiwan and also with the Research Center for Information Technology Innovation, Academia Sinica, Taipei 115, Taiwan (e-mail: ywchang@ntu.edu.tw).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TCAD.2014.2360453

I. INTRODUCTION

DUE to the advancement of process technology, a modern circuit design could contain more than billions of transistors in a single chip. Although the placement problem has been studied for several decades, modern placement still remains very tough due to the fast-growing design complexity and advanced process technologies. Therefore, many new placement algorithms were developed recently, and most of these algorithms try to optimize total wirelength for better circuit performance. However, because existing wirelength models cannot accurately model the congestion objective in placement, a wirelength-driven placer that does not consider routability could introduce irresolvable routing-congested placements. Furthermore, the big difference of large macros and small standard cells brings new challenges to modern mixed-size placement. Because large macros usually occupy several metal layers (called macro porosity), routing resources for a modern mixed-size circuit design are more constrained than traditional circuit designs.

In recent years, hierarchical design methodologies are often employed in modern circuit designs for faster turnaround time [3]. While placing circuit elements which belong to a particular hierarchy closely is expected to have better routability and circuit performance [3], [4], how to honor the design hierarchy of a circuit during placement incurs big challenges to modern circuit designs. Therefore, it is desirable to develop an effective routability-driven placer for modern hierarchical mixed-size designs.

A. Previous Work

The routability-driven placement problem has attracted increasing attention in recent literature. Congestion-driven Dragon [5] first distributes whitespace into rows and then into bins within a row. As it may increase the row imbalance, a lower bound and an upper bound on the amount of whitespace available in a row are imposed. Li *et al.* [6] distributes whitespace by adjusting the cut-lines of hierarchically sliced placement and replacing cells to avoid congested regions. ROOSTER [7] uses rectilinear spanning minimal trees as its routing congested model during global placement and applies whitespace allocation during detailed placement. BonnPlace [8], CRISP [9], Ripple [10], [11], and SimPLR [12], [13] implicitly allocate whitespace by inflating

cell area according to estimated routing congestions. Although allocating whitespace is pervasive in fixed-die placement, inappropriate whitespace distribution might induce longer wirelength and hurt circuit routability. Instead of allocating whitespace during placement, RUDY [14] estimates routing congestion and modifies the density term to consider both the routing density and module density. Chuang *et al.* [3] and Jiang *et al.* [15] reduce congestion by removing overlaps between net bounding boxes. In addition to reducing overlaps between net bounding boxes, Chuang *et al.* [3] utilizes design-hierarchy information to further improve circuit routability. Tsota *et al.* [16] integrates the wire density term into an analytical placement framework. Incorporating routability models in the optimization objective during placement could effectively optimize routability. However, how to accurately estimate the exact effect of routing incurs big challenges.

B. Our Contributions

This paper presents a new routability-driven analytical placement algorithm for hierarchical mixed-size circuit designs. As preserving design hierarchies physically is significantly important for modern circuit designs, we propose a novel design hierarchy identification technique to effectively identify design hierarchies and guide placement for better wirelength and routability. Furthermore, unlike most existing works which usually optimize routability by implicit or explicit reallocating whitespace, we propose a new routability-driven placement algorithm which optimizes circuit routability from four major aspects: 1) narrow channel handling; 2) pin density; 3) routing overflow optimization; and 4) net congestion optimization. We summarize the main contributions of our proposed algorithm as follows.

- 1) A novel design hierarchy identification technique is proposed to effectively identify design hierarchies and guide placement for better wirelength and routability.
- 2) We introduce a novel multilayer sigmoid potential model that considers both global and local block densities to facilitate the global spreading during analytical placement.
- 3) We propose a smoothed congestion estimation method to model the global routing behavior and try to give more accurate congestion estimation during placement.
- 4) To consider the constrained routing resource incurred by big macros, a new narrow-channel-handling technique is proposed.
- 5) A pin density control technique is proposed. Different from most of existing works which optimize routability according to net distribution, the density of pins, and the routing directions of the pins are considered during our placement formulation.
- 6) A novel sigmoid function-based overflow refinement method for analytical global placement is presented. Different from previous works which usually use whitespace allocation to improve routability, routing overflow is formulated by a sigmoid function and optimized during global placement.

- 7) A novel net-congestion-optimization technique is proposed to optimize routability and wirelength simultaneously during placement.

The remainder of this paper is organized as follows. Section II formulates the routability-driven placement problem, and gives a brief overview of our analytical placement framework. Section III introduces our proposed routability-driven placement algorithm, and the details of the algorithm are presented in Sections IV–VII. Experimental results are shown in Section VIII.

II. PRELIMINARIES

In this section, we introduce the routability-driven placement problem and our analytical placement framework.

A. Problem Formulation

The circuit placement problem can be formulated as a hypergraph $H = (V, E)$ placement problem. Let vertices $V = \{v_1, v_2, \dots, v_n\}$ represent blocks and hyperedges $E = \{e_1, e_2, \dots, e_m\}$ represent nets. Let x_i and y_i be the x and y coordinates of the center of block v_i , respectively. The blocks to be placed can be categorized into two types: pre-placed blocks and movable blocks. Preplaced blocks have fixed x and y coordinates and cannot be moved.

Global routing is often used to evaluate the routability of a placement. After dividing the routing region into uniform and nonoverlapping grids, a routing edge is the edge between two adjacent routing grids, and the capacity of each routing edge is defined by the available routing resources on the edge. Then, the circuit routability can be evaluated by the total overflow of all routing edges. It should be noted that, the capacities of routing edges which overlap with big macros would decrease due to the macro porosity issue. Moreover, to model local congestions during global routing, pin density in each routing grid is usually considered. In [17], to model local congestions more precisely, pin densities are formulated as routing blockages, and thus will also decrease the capacities of routing edges. The routability-driven placement can then be defined as follows. Given a netlist and global routing resources, we intend to determine the desired positions of movable blocks so that the total wirelength and total overflow are minimized.

B. Our Analytical Placement Framework

The placement problem is usually solved in three stages: 1) global placement; 2) legalization; and 3) detailed placement. Global placement evenly distributes the blocks and finds the best position for each block while minimizing the target cost (e.g., wirelength). Then legalization removes all overlaps. Finally, detailed placement refines the placement solution. Generally, global placement has the most crucial impact on the overall placement quality, and is thus considered as the most critical step [18].

After the placement region is divided into uniform nonoverlapping grids, the global placement problem can be formulated as a constrained minimization problem as follows:

$$\begin{aligned} &\min W(\mathbf{x}, \mathbf{y}) \\ &\text{s.t. } D_b(\mathbf{x}, \mathbf{y}) \leq M_b, \text{ for each bin } b \end{aligned} \quad (1)$$

where $W(\mathbf{x}, \mathbf{y})$ is the wirelength function, $D_b(\mathbf{x}, \mathbf{y})$ is the potential function that is the total area of movable blocks in bin b , and M_b is the maximum allowable area of movable blocks in bin b . M_b can be computed by $M_b = t_{\text{density}}(w_b h_b - P_b)$, where t_{density} is a user-specified target density value for each bin, w_b (h_b) is the width (height) of bin b , P_b is the base potential equal to the preplaced block area in bin b , and M_b is a fixed value as long as all preplaced block positions are given and the bin size is determined.

During placement, the wirelength $W(\mathbf{x}, \mathbf{y})$ is usually defined as the total half-perimeter wirelength (HPWL)

$$W(\mathbf{x}, \mathbf{y}) = \sum_{e \in E} \left(\max_{v_i, v_j \in e} |x_i - x_j| + \max_{v_i, v_j \in e} |y_i - y_j| \right) \\ = \sum_{e \in E} \left(\max_{v_i \in e} x_i - \min_{v_i \in e} x_i + \max_{v_i \in e} y_i - \min_{v_i \in e} y_i \right). \quad (2)$$

Since $W(\mathbf{x}, \mathbf{y})$ in (2) is not smooth, it is hard to minimize it directly. As a result, several smooth wirelength approximation functions have been proposed. We use the weighted-average wirelength model proposed in [19] to approximate the HPWL in (2).

The potential function is defined as

$$D_b(\mathbf{x}, \mathbf{y}) = \sum_{v \in V} P_x(b, v) P_y(b, v) \quad (3)$$

where P_x and P_y are the overlap functions of bin b and block v along the x and y directions. Since $D_b(\mathbf{x}, \mathbf{y})$ is neither smooth nor differentiable, the sigmoid potential model [20] can be used to smooth the density for each block.

Equation (1) can be solved by the quadratic penalty method, implying that we solve a sequence of unconstrained minimization problems of the form

$$\min \hat{W}(\mathbf{x}, \mathbf{y}) + \lambda \sum_b (\hat{D}_b(\mathbf{x}, \mathbf{y}) - M_b)^2 \quad (4)$$

with increasing λ 's, where $\hat{W}(\mathbf{x}, \mathbf{y})$ and $\hat{D}_b(\mathbf{x}, \mathbf{y})$ are the smoothed wirelength and density functions. The solution of the previous problem is used as the initial solution for the next one. We solve the unconstrained problem in (4) by the nonlinear conjugate gradient (CG) method.

III. PROPOSED ALGORITHM FLOW

In this paper, we propose a new routability-driven analytical placement algorithm. Based on the analytical placement framework in Section II-B, the proposed algorithm consists of three main stages: 1) routability-driven global placement with routing overflow minimization; 2) routability-driven legalization with routing congestion awareness; and 3) routability-driven detailed placement. Fig. 1 summarizes the overall flow of the proposed algorithm.

In routability-driven global placement, we optimize the routability from four major aspects.

- 1) *Narrow Channel Handling*: A new narrow-channel-handling technique is applied to consider the constrained routing resource incurred by big macros.
- 2) *Pin Density*: Our algorithm considers both the density of pins and their routing directions.

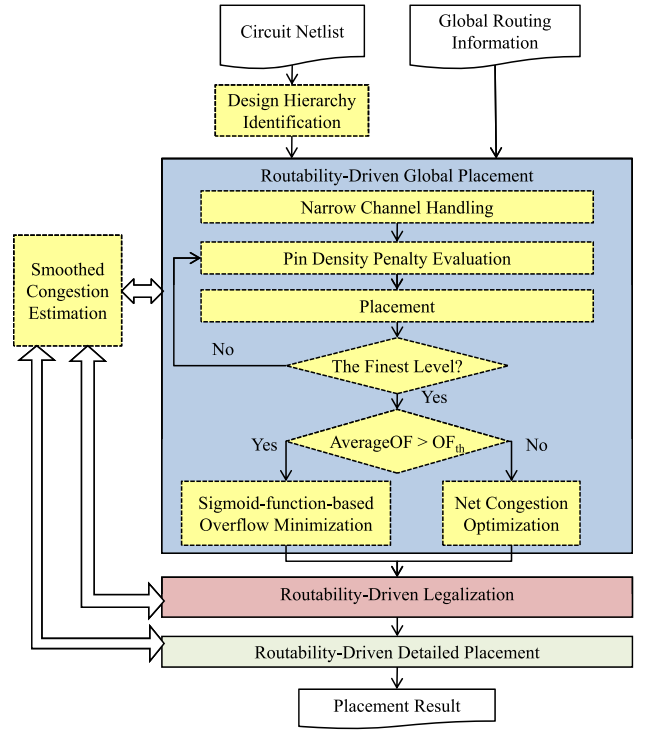


Fig. 1. Overview of our proposed routability-driven placement flow.

- 3) *Routing Overflow Optimization*: Our algorithm optimizes routing overflow by a novel sigmoid function during global placement.
- 4) *Net Congestion Optimization*: Our algorithm optimizes net congestions while minimizing wirelength.

During global placement, the multilevel framework [21], [22] is used to improve scalability. In the legalization stage, in addition to legalizing cells with minimum displacements, routing congested regions are identified and considered for routability improvement. In the detailed placement stage, cell matching, and cell swapping techniques are applied.

Different from most of existing detailed placement techniques, routing congestions are modeled as costs and optimized. Furthermore, a novel design hierarchy identification technique is proposed to effectively identify design hierarchies and guide placement for better wirelength and routability, and during each stage, we adopt a smoothed congestion estimation method to model the global routing behavior and try to give more accurate congestion estimation during placement. Fig. 2 shows our proposed routability-driven analytical placement algorithm. Line 1 is the design hierarchy identification. Lines 2–5 are the coarsening stage. The initial placement is generated by quadratic placement [23] in line 6. Lines 7–22 are the uncoarsening stage. Lines 23 and 24 are routability-driven legalization and routability-driven detailed placement, respectively. We detail each stage in the following sections.

IV. DESIGN HIERARCHY IDENTIFICATION

Preserving design hierarchies physically during placement is expected to guide the placement to minimize wirelength as well as routing congestion [3], [4]. However, inappropriate

Algorithm: Routability-Driven Analytical Placement**Input:**hypergraph H_0 : circuit to be placed n_{max} : the maximum block number in the coarsest level**Output:** (x^*, y^*) : optimal block positions

```

01. call DesignHierarchyIdentification
02.  $level = 0$ ;
03. while ( $BlockNumber(H_{level}) > n_{max}$ )
04.    $level++$ ;
05.    $H_{level} = FirstChoiceClustering(H_{level-1})$ ;
06. initialize block positions by QuadraticPlacement( $H_{level}$ );
07. call NarrowChannelHandling;
08. for  $currentLevel = level$  to 0
09.   initialize bin grid size  $n_{bin} \propto \sqrt{n_x}$ ;
10.   call SmoothedCongestionEstimation();
11.   initialize  $\lambda_0 = \frac{\sum |\partial \hat{W}(\mathbf{x}, \mathbf{y})|}{\sum |\partial \hat{D}_b(\mathbf{x}, \mathbf{y})|}$ ;  $m = 0$ ;
12.   do
13.     call PinDensityPenaltyEvaluation();
14.     solve  $\min \hat{W}(\mathbf{x}, \mathbf{y}) + \lambda_m \sum_b \max(\hat{D}_b(\mathbf{x}, \mathbf{y}) - \hat{M}_b, 0)^2$ ;
15.      $m++$ ;
16.      $\lambda_m = 2\lambda_{m-1}$ ;
17.   until (spreading enough or
           no further improvement in overflow)
18.   decluster and update block positions;
19. if ( $AverageOF > OF_{th}$ )
20.   call OverflowOptimization();
21. else
22.   call NetCongestionOptimization();
23.   call RoutabilityDrivenLegalization();
24.   call RoutabilityDrivenDetailedPlacement();

```

Fig. 2. Our routability-driven analytical placement algorithm.

handling of design hierarchies might hurt the wirelength and routability substantially. As mentioned before, our algorithm adopts the multilevel framework to improve the scalability. While the initial block positions are determined after the coarsening stage, clustering blocks with deeper hierarchical relations could help to preserve design hierarchies during the uncoarsening stage and thus guide the placement for better wirelength and routability. Therefore, in this section, we propose a design hierarchy identification technique which includes: 1) macro-based hierarchy grouping (HG) and 2) HG-based clustering to identify design hierarchies and guide placement for better wirelength and routability. In the macro-based HG step, we construct a set of HGs based on the design hierarchy of macros, where an HG is a group of blocks which are candidates for clustering. Then, we cluster blocks based on these HGs. By using HGs, blocks could be clustered efficiently while considering the design hierarchy for better placement results. We detail each step in the following subsections.

A. Macro-Based HG

From designers' perspective, macros and the standard cells which have strong connectivity with those macros would be seen as in the same hierarchy. In modern mixed-size circuit designs [4], most macros are preplaced and fixed, and thus clustering blocks only based on design hierarchy information [3] and without considering the physical information of macros might not be sufficient. Therefore, in our design hierarchy identification method, we determine a set of HGs based on the hierarchy and positions of macros for better clustering quality.

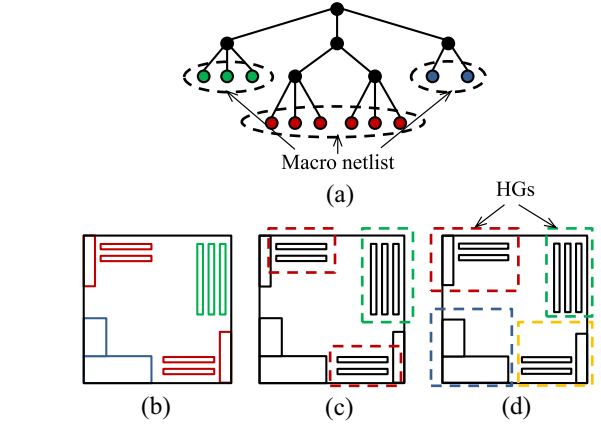


Fig. 3. Illustration of macro-based HG. (a) MHST of a set of macros. (b) Corresponding placement of these macros. (c) Grouping of neighboring macros with the same dimension. (d) Four HGs constructed according to the MHST. Note that the macros on the top-left corner and bottom-right corners should not be identified as the same HG because the distance of them is too large.

In our macro-based HG method, we first construct macro hierarchical structure trees (MHSTs) according to the design hierarchy of macros. Fig. 3(a) shows an example of an MHST for a set of macros, and the corresponding placement of these macros is shown in Fig. 3(b). After the construction of an MHST, macros in the MHST with the same dimension and placed close to each other are grouped first, as shown in Fig. 3(c), and other macros are then grouped in a bottom-up manner according to the MHST, as shown in Fig. 3(d). As mentioned before, the positions of preplaced macros should be considered carefully during clustering. As shown in Fig. 3(d), the macros on the top-left and bottom-right corners belong to the same hierarchy, but they should not be identified as the same HG because their distance is too large. Hence, during constructing the HGs, we should consider not only the design hierarchy information, but also the distance between two macros. In our implementation, two macros should not be grouped if the distance between the centers of these two macros is larger than the width/height of the corresponding placement grid. To finish the construction of HGs, for each macro, we cluster the standard cells with the same hierarchy as that of the macros into the same group. For standard cells which do not have the same hierarchy as any macro, they are grouped according to the original design hierarchy information [3].

B. HG-Based Clustering

To simultaneously consider the design hierarchy and net connectivities during the coarsening stage, we propose an HG-based clustering to evaluate the affinity between two blocks which denotes the relation between the two blocks. We attempt to cluster blocks with higher affinities earlier. Unlike the score function of clustering in [3], the HGs are introduced to emphasize the effect of the design hierarchy of macros, and the affinity value Φ is computed as follows:

$$\Phi(v_i, v_j) = \frac{\Gamma_{i,j}}{|e_{i,j}| \cdot |a_i + a_j|} \quad (5)$$

where $|e_{i,j}|$ is the number of terminals of a net connecting the two blocks v_i and v_j , and a_i and a_j denote the respective areas of blocks v_i and v_j . The hierarchy factor $\Gamma_{i,j}$ is defined as

$$\Gamma_{i,j} = \begin{cases} \epsilon \cdot k_{i,j}, & \text{if } v_i, v_j \text{ are in the same HG} \\ k_{i,j}, & \text{others} \end{cases} \quad (6)$$

where $k_{i,j}$ is the common naming factor which is defined as the number of common hierarchy parts between the two blocks v_i and v_j . In a hierarchical design, the name of a block usually consists of several hierarchy parts to represent its hierarchy [4], such as A/B/C/D, and the common hierarchy parts of two blocks means the common part of these two blocks' name. A large $k_{i,j}$ implies that these two blocks have a high hierarchy relation. The user-defined parameter ϵ denotes that two blocks have a larger $\Gamma_{i,j}$ if they are in the same HG. In our implementation, we set ϵ equal to 10.

V. ROUTABILITY-DRIVEN GLOBAL PLACEMENT

In this section, we propose a routability-driven analytical global placement algorithm. We optimize the routability from four aspects: 1) narrow channel handling; 2) pin density; 3) routing overflow optimization; and 4) net congestion optimization. Since in the upper level of the global placement, cells are grouped, it is hard to estimate the routing topologies. Instead of considering routing nets, pin density is formulated and optimized in the analytical global placement formulation in (4). At the finest level of the global placement with known block distribution, routing congestions are estimated, and average overflow per G-cell is evaluated. If the average overflow is larger than a threshold, e.g., 50% larger than the average capacities of routing edges, routing overflows are formulated through a sigmoid function and optimized as a nonlinear optimization; else, we optimize net congestions with wire-length consideration. To deal with the macro porosity issue, a new narrow channeling handling technique is also proposed. Furthermore, for efficient block spreading, we present a multilayer sigmoid potential model which considers both global and local block densities, and a smoothed congestion estimation method to model the global routing behavior and try to give more accurate congestion estimation during placement. We detail each technique as follows.

A. Multilayer Sigmoid Potential Model

As mentioned in Section II-B, since the density function in (3) is neither smooth nor differentiable, several approximation functions are proposed to smooth $P_x(b, v)$ and $P_y(b, v)$, such as the bell-shaped potential model [24] and the sigmoid potential model [20]. Compared with the bell-shaped smoothing function, the sigmoid smoothing function gives more accurate approximation to the original overlap function as illustrated in Fig. 4. However, while the range of block's potential for the sigmoid smoothing function is $w_b + w_b$ ($h_v + h_b$) in the x (y) direction which is smaller than that for the bell-shaped smoothing function, the sigmoid smoothing might not spread blocks into the placement region efficiently, compared with the bell-shaped smoothing, especially for sparse designs. To handle the global spreading

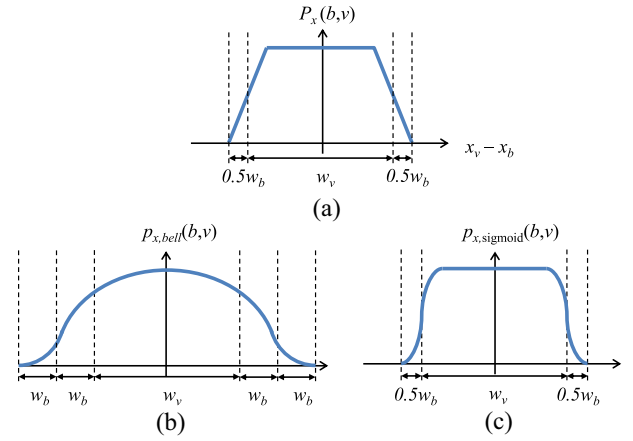


Fig. 4. Density Models. (a) Overlap function $P_x(b, v)$. (b) Bell-shaped smoothing function $P_{x,bell}(b, v)$. (c) Sigmoid smoothing function $P_{x,sigmoid}(b, v)$.

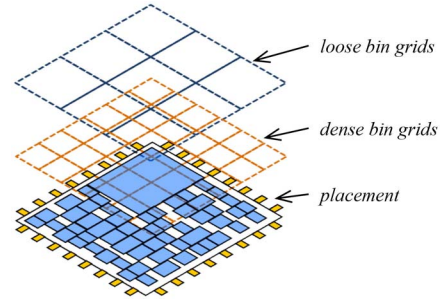


Fig. 5. Illustration of using two layers of bin grids for our multilayer sigmoid potential model. While the dense bin grids are used to consider local densities, the loose bin grids are used to handle global densities. As a result, our model can help spreading blocks into the placement region efficiently.

problem, we introduce a novel multilayer sigmoid potential model which considers both global and local block densities.

Instead of using a single layer of bin grids for density computation, our multilayer sigmoid potential model uses multiple layers of bin grids with different sizes during global placement. Fig. 5 illustrates using multiple layers of bin grids for density computation. Our multilayer sigmoid potential model applies two layers: one layer with loose bin grids and the other layer with dense bin grids. While the dense bin grids are used to consider local densities, the loose bin grids are used to handle global densities. As a result, our model can help spread blocks into the placement region efficiently. We will validate the effectiveness of our multilayer sigmoid potential model in Section VIII-A.

B. Smoothed Congestion Estimation

To estimate the routing congestion, the L-shaped probabilistic model is one of the most popular models for placement [14], where each 2-pin net is routed by upper-L and lower-L patterns with 50% probability for each direction. However, the L-shaped probabilistic model might not accurately capture global routing behavior, such as routing detours and rip-up and reroute during global routing. To model the global routing behavior more precisely, we apply

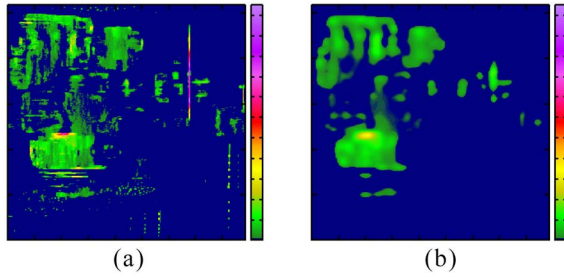


Fig. 6. Illustration of smoothed congestion estimation. (a) Original congestion estimation. (b) Congestion estimation after Gaussian smoothing.

Gaussian smoothing [25] to smooth the L-shaped approximation model. In the 2-D case, a circularly symmetric Gaussian has the form

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (7)$$

where σ is the standard deviation of the Gaussian distribution. Fig. 6 shows an example of our smoothed congestion estimation. Fig. 6(a) shows an original congestion estimation, and Fig. 6(b) illustrates the congestion estimation after Gaussian smoothing.

C. Narrow Channel Handling

As mentioned in Section I, the limited routing resource induced by big macros make the routability-driven placement problem more difficult. Due to the macro porosity issue, many routing overflows occur along macro boundaries, especially for narrow channels between macros. Narrow channels are formed between two or more close fixed macros, and placing too many blocks in these narrow channels would incur significant routing overflows.

To reduce the congestion caused by narrow channels, one intuitive method is to block these channels during placement. However, this method might lack flexibility and overkill placement solutions because either it blocks all the narrow channels or it does not block them at all. There is also an existing technique which applies Gaussian smoothing to handle preplaced macros [22]. However, directly smoothing the density of big macros could still induce deep ravines between neighboring macros which trap movable blocks during placement and limit the final placement quality. To solve this problem, we propose a narrow-channel-handling technique which adaptively modifies the density constraints around narrow channels to reduce routing congestions.

In our narrow-channel-handling technique, we first detect the locations of narrow channels, and then modify the maximum allowable area in each bin to prevent blocks from getting stuck in the narrow channels during global placement. As mentioned in Section II-B, the maximum allowable area in each bin b is defined as $M_b = t_{\text{density}}(w_b h_b - P_b)$, where t_{density} is a user-specified target density value for each bin, w_b (h_b) is the width (height) of bin b and P_b is the base potential equal to the preplaced block area in bin b . We define P'_b as the narrow-channel-aware base potential in bin b

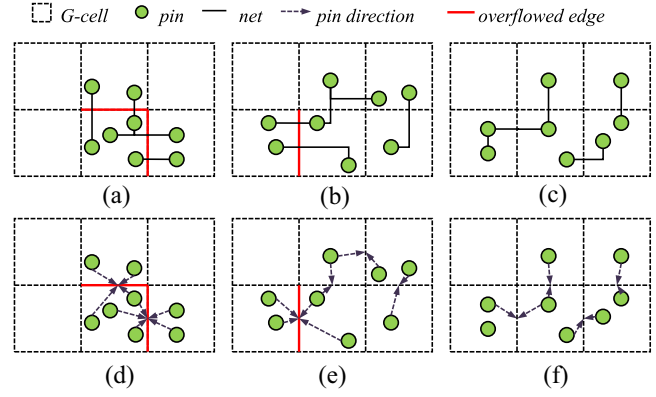


Fig. 7. Illustration of pin density control. Assume that the capacity of each routing edge equals one. (a) Placement with wirelength minimized. Routing overflows occur due to high pin density in G-cells. (b) Placement with evenly distributed pins. Routing overflows are reduced; however, since two nets go through the same routing edge, an overflow occurs. (c) Placement with evenly distributed pins and pin direction consideration. Since congestions on routing edges are calculated according to the pin routing directions, routing overflow can be estimated more accurately and minimized more effectively. (d)–(f) Illustration of pin routing directions of the placements in (a)–(c).

after considering narrow channels

$$P'_b = \begin{cases} P_b \left(\beta + (1 - \beta) \left(1 - \left(\frac{P_b}{w_b h_b} - \beta \right)^2 \right) \right), & \text{if } \frac{P_b}{w_b h_b} \geq \beta \\ P_b, & \text{others} \end{cases} \quad (8)$$

where β is a base potential threshold such that if $P_b/w_b h_b$ is greater than β , then bin b is treated as a narrow channel. We set β equal to 0.6 in our implementation. We then smooth the base potential by Gaussian smoothing [25] to make the density model differentiable. The detailed formulation for the Gaussian function has been introduced in Section V-B. With first increasing the base potential at narrow channels by using (8), our proposed narrow-channel-handling technique could prevent blocks from being trapped in narrow channels and achieve better placement quality.

D. Pin Density Penalty Evaluation

In this section, we introduce a pin density control method during the analytical global placement. Different from most previous works which only optimize net congestion during placement, pin density in each G-cell is evaluated and optimized to improve routability. While pins are connected through nets, a large number of pins in a G-cell will cause high routing congestion. Fig. 7 illustrates the importance of pin density consideration during placement. If pin density is not considered, routing overflow occurs due to the high pin density in G-cells [see Fig. 7(a)], and routing congestion could be reduced by distributing pins evenly in the G-cells. It should be noted that, in addition to the number of pins, the routing directions of pins should be also considered. Fig. 7(b) shows an example. Although pins are distributed evenly, overflow still occurs since there are two nets going through the same routing edge (assume that the capacity of each routing edge equals one). To overcome this problem, during pin density

computation, pins with different directions should be considered independently, as shown in Fig. 7(d) and (e). Fig. 7(f) shows an example of placement considering both pin density and pin routing directions.

To control the total number of pins in a G-cell, we let the dimension of each placement bin be the same as that of G-cells, and formulate pin density penalties in the density constraints in (4). Assume that $p_b(\mathbf{x}, \mathbf{y})$ is the pin density in bin b , which is the ratio between the total number of pins in b and the total number of allowed pins in each bin, then the pin density penalty is defined

$$N_b(\mathbf{x}, \mathbf{y}) = p_b(\mathbf{x}, \mathbf{y})A_f \quad (9)$$

where A_f is the total movable area for placement. In our implementation, the total number of allowed pins in each bin is assumed to be the average number of pins in each bin. By subtracting the maximum potential of a bin b by its pin density penalty, we can get the revised density constraints for placement as follows:

$$D_b(\mathbf{x}, \mathbf{y}) \leq \hat{M}_b, \text{ for each bin } b \quad (10)$$

where c_b is used to normalize the density penalty so that the summation of density penalties is equal to the total movable area A_f , and $\hat{M}_b = (M_b - c_b N_b(\mathbf{x}, \mathbf{y}))$.

With the pin density penalty, the maximum allowed area of movable blocks in a bin b will be reduced when the pin density in b is high. As a result, pins can be distributed evenly, and thus the optimized placement result can facilitate global routing. It should be noted that, to consider pins with different directions, nets are first routed by the L-shaped probabilistic routing as mentioned in Section V-B, and pin density of a G-cell is the summation of pin densities on its corresponding routing edges.

E. Overflow Optimization

In this section, we introduce two methods for improving the routability of global placement results: overflow refinement and congestion removal. While overflow refinement optimizes the overflow globally by using a nonlinear formulation, congestion removal locally spreads blocks in congested regions. We detail these two methods as follows.

1) *Overflow Refinement*: For overflow refinement, we propose a nonlinear formulation based on L-shaped probabilistic routing, which computes the expected usage of routing edges, i.e., the number of wires passing through routing edges. Since originally the routing method computes the expected usage in a discrete fashion, we propose to use a sigmoid function to make the usage smooth and differentiable, such that we can leverage the analytical framework in Section III to optimize the routability. Since our formulation is based on L-shaped routing, we decompose each multipin net into two-pin nets by FLUTE [26] as mentioned in Section V-B. Then, we optimize the overflow by solving a constrained minimization problem of the form

$$\begin{aligned} \min \quad & W(\mathbf{x}, \mathbf{y}) \\ \text{s.t.} \quad & D_b(\mathbf{x}, \mathbf{y}) \leq M_b, \text{ for each bin } b \\ & E_e(\mathbf{x}, \mathbf{y}) \leq C_e, \text{ for each routing edge } e \end{aligned} \quad (11)$$

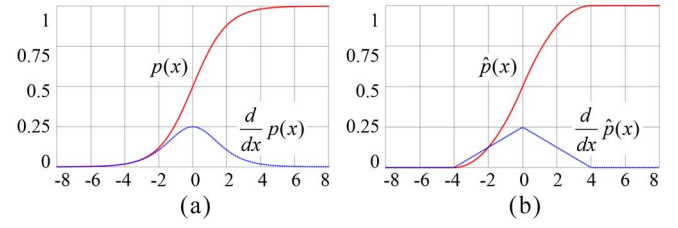


Fig. 8. Sigmoid functions. Illustration of the (a) logistic function with $\alpha = 1$ and (b) quadratic sigmoid function with $\alpha = 0.125$.

where E_e denotes the expected usage of routing edge e . C_e denotes the routing capacity of the routing edge. Note that E_e is a function of block positions.

In order to represent the equation of E_e in terms of block positions, we first define a 0-1 logic function $f(l, x, u)$ as follows:

$$f(l, x, u) = \begin{cases} 1, & \text{if } l < x < u \\ 0, & \text{otherwise.} \end{cases} \quad (12)$$

For a vertical edge from (x_e, y_e^b) to (x_e, y_e^t) , its expected usage is defined as

$$\sum_n \left(0.5 \cdot (f(x_{p_1}(n), x_e, x_{p_2}(n)) + f(x_{p_2}(n), x_e, x_{p_1}(n))) \cdot (f(y_e^b, y_{p_1}(n), y_e^t) + f(y_e^b, y_{p_2}(n), y_e^t)) \right) \quad (13)$$

where $(x_{p_1}(n), y_{p_1}(n))$ and $(x_{p_2}(n), y_{p_2}(n))$ are the coordinates of the connected pins of a two-pin net n . The expected usage of a horizontal edge can be defined in a similar way.

Since the 0-1 logic function $f(l, x, u)$ is neither smooth nor differentiable, we propose to use a sigmoid function to make the function differentiable such that the probabilistic method can be integrated into our analytical framework. The most commonly used sigmoid function is logistic function shown as follows:

$$p(x) = \frac{1}{1 + e^{-\alpha x}} \quad (14)$$

where α is the parameter for controlling the rate of growth. Fig. 8(a) illustrates the logistic function and its derivative.

It should be noted that, although the sigmoid function can be used to smooth the expected usage function, the computation of the logistic function involves the function call of exponential subroutines, which is a quite expensive operation in practice. Therefore, we propose to use a quadratic sigmoid function in our analytical framework

$$\hat{p}(x) = \begin{cases} 1, & 0.5 \leq \alpha x \\ 1 - 2(\alpha x - 0.5)^2, & 0 \leq \alpha x \leq 0.5 \\ 2(\alpha x + 0.5)^2, & -0.5 \leq \alpha x \leq 0 \\ 0, & \alpha x \leq -0.5. \end{cases} \quad (15)$$

Theorem 1: The quadratic sigmoid function $\hat{p}(t)$ in (15) is continuously differentiable.

Proof: By differentiating (15) with respect to x , we can get

$$\frac{d}{dx}\hat{p}(x) = \begin{cases} 0, & 0.5 \leq \alpha x \\ -4\alpha(x - 0.5), & 0 \leq \alpha x \leq 0.5 \\ 4\alpha(x + 0.5), & -0.5 \leq \alpha x \leq 0 \\ 0, & \alpha x \leq -0.5. \end{cases} \quad (16)$$

Thus, $\hat{p}(x)$ is continuously differentiable. ■

Fig. 8(b) illustrates the quadratic sigmoid function and its derivative.

With the quadratic sigmoid function, we can define the smoothed 0-1 logic function as follows:

$$\hat{f}(l, x, u) = \hat{p}(x - l)\hat{p}(u - x) \quad (17)$$

where $\hat{p}(x - l)$ transforms the condition $l < x$ while $\hat{p}(u - x)$ transforms the condition $x < u$. Then, we have the following theorem.

Theorem 2: The smoothed 0-1 logic function $\hat{f}(l, x, u)$ is continuously differentiable.

Proof: Since

$$\frac{d}{dx}\hat{f}(l, x, u) = \left(\frac{d}{dx}\hat{p}(x - l)\right)\hat{p}(u - x) + \hat{p}(x - l)\left(\frac{d}{dx}\hat{p}(u - x)\right) \quad (18)$$

and $\hat{p}(x)$ is continuously differentiable, the smoothed 0-1 logic function $\hat{f}(l, x, u)$ is continuously differentiable. ■

To solve (11), we use the quadratic penalty method to transform the constrained optimization problem into an unconstrained problem. Our overall nonlinear formulation is as follows:

$$\begin{aligned} \min \quad & \lambda_1 \hat{W}(\mathbf{x}, \mathbf{y}) + \lambda_2 \sum_b \max(\hat{D}_b(\mathbf{x}, \mathbf{y}) - M_b, 0)^2 \\ & + \lambda_3 \sum_e \max(\hat{E}_e(\mathbf{x}, \mathbf{y}) - C_e, 0)^2 \end{aligned} \quad (19)$$

where λ_1 , λ_2 , and λ_3 are weights for wirelength, density, and routability, respectively, and $\hat{E}_e(\mathbf{x}, \mathbf{y})$ is the smoothed expected usage function of a routing edge e . In our implementation, we set $\lambda_1 = \|\nabla \hat{E}(\mathbf{x}, \mathbf{y})\|/\|\nabla \hat{W}(\mathbf{x}, \mathbf{y})\|$, $\lambda_2 = \|\nabla \hat{E}(\mathbf{x}, \mathbf{y})\|/\|\nabla \hat{D}(\mathbf{x}, \mathbf{y})\|$, and $\lambda_3 = 1$, and set α of the sigmoid function to the reciprocal of the G-cell size.

2) *Congestion Removal:* The congestion removal method consists of three steps: 1) congested region identification; 2) adaptive base potential modification; and 3) nonlinear optimization. In congested region identification, we calculate the routing overflow of each bin based on the congestion estimation in Section V-B. We then modify the base potential P_b of every congested bin adaptively. If the overflow of a bin is smaller than the average, we slightly reduce the base potential. On the contrary, if the overflow of a bin is larger than the average, we reduce the base potential more aggressively. In our implementation, the congestion-aware base potential of a bin b is defined as

$$\begin{aligned} \hat{P}_b = & \max(w_b h_b - (0.5 \cdot D_b(\mathbf{x}, \mathbf{y}) \\ & + 0.5 \cdot (1 - g_b(\mathbf{x}, \mathbf{y})) \cdot D_b(\mathbf{x}, \mathbf{y})) 0) \end{aligned} \quad (20)$$

where $g_b(\mathbf{x}, \mathbf{y})$ is the average overflow within b . Besides, we apply Gaussian smoothing [25] to smooth the base potential

of bins. Finally, in nonlinear optimization, we optimize the objective in (4) subject to the congestion-aware base potentials. In this way, we can spread blocks in congested regions while still trying to minimize the total wirelength. We apply the congestion removal method iteratively until there is no further improvement in routing overflow. It should be noted that, instead of using the pin density penalty to evenly spread pins in Section V-D, routing overflow is used to guide the following placement process to locally remove routing congested regions for better routability.

F. Net Congestion Optimization

Because minimizing routed wirelength is always an important objective for practical applications, we propose a new net congestion optimization technique considering wirelength. Different from RUDY [14] and Tsota *et al.* [16] which estimate routing congestion and modify the density term to consider both the routing density and module density, we optimize net congestion directly based on a nonlinear formulation. In particular, instead of using the quadratic [14] or CHKS [16] wirelength approximations, we adopt the weighted-average wirelength approximation [19] to smooth the horizontal and vertical overlap lengths between nets and placement bins. In the following, we detail our net congestion optimization technique.

We optimize the net congestion by solving a constrained minimization problem of the form

$$\begin{aligned} \min \quad & W(\mathbf{x}, \mathbf{y}) \\ \text{s.t.} \quad & D_b(\mathbf{x}, \mathbf{y}) \leq M_b, \text{ for each bin } b \\ & C_b(\mathbf{x}, \mathbf{y}) \leq S_b, \text{ for each bin } b \end{aligned} \quad (21)$$

where $C_b(\mathbf{x}, \mathbf{y})$ denotes the net congestion of bin b , and S_b is defined as the total allowable routing area in bin b . Note that $C_b(\mathbf{x}, \mathbf{y})$ is a function of block positions, and S_b would decrease due to the macro porosity issue.

To evaluate the net congestion of bin b , we first define the weighted perimeter-per-area (p/a) ratio of a net e as follows:

$$d_e(\mathbf{x}, \mathbf{y}) = \frac{L_{x,e}(\mathbf{x}, \mathbf{y}) \cdot \sigma_v + L_{y,e}(\mathbf{x}, \mathbf{y}) \cdot \sigma_h}{L_{x,e}(\mathbf{x}, \mathbf{y}) \cdot L_{y,e}(\mathbf{x}, \mathbf{y})} \quad (22)$$

where $L_{x,e}$ and $L_{y,e}$ are the respective horizontal and vertical bonding box lengths of net e , and σ_v and σ_h are the respective average vertical and horizontal wire pitches. The average vertical (horizontal) pitch could be obtained by dividing the summation of vertical (horizontal) pitches in every routing layers by the total number of routing layers. As shown in (22), while minimizing $d_e(\mathbf{x}, \mathbf{y})$, a net with smaller HPWL, i.e., $L_{x,e} + L_{y,e}$, and square bounding box is preferred.

The overlaps between a net e and a bin b can be computed by

$$\phi_{b,e}(\mathbf{x}, \mathbf{y}) = o_h(b, e) \cdot o_v(b, e) \quad (23)$$

where $o_h(b, e)$ and $o_v(b, e)$ are the horizontal and vertical overlaps between net e and bin b . With the net density defined in (22), the net congestion of a bin b is then defined as follows:

$$C_b(\mathbf{x}, \mathbf{y}) = \sum_e (d_e(\mathbf{x}, \mathbf{y}) \cdot \phi_{b,e}(\mathbf{x}, \mathbf{y})). \quad (24)$$

As we can see in (24) the net congestion $C_b(\mathbf{x}, \mathbf{y})$ consists of two major parts: 1) overlaps between nets and bins and 2) the weighted p/a ratio of each net. So, if we minimize $C_b(\mathbf{x}, \mathbf{y})$, both the net lengths and overlaps between nets and bins are expected to be reduced.

We then use the quadratic penalty method to solve (21) by transforming the constrained optimization problem into an unconstrained problem as follows:

$$\min \lambda_1 \hat{W}(\mathbf{x}, \mathbf{y}) + \lambda_2 \sum_b \left(\hat{D}_b(\mathbf{x}, \mathbf{y}) - M_b \right)^2 + \lambda_3 \sum_e \left(\hat{C}_b(\mathbf{x}, \mathbf{y}) - S_b \right)^2 \quad (25)$$

where λ_1 , λ_2 , and λ_3 are weights for wirelength, density, and net congestion, respectively, and $\hat{C}_b(\mathbf{x}, \mathbf{y})$ is the smoothed net congestion of a bin b . Since both the net lengths ($L_{x,e}$ and $L_{y,e}$) and the overlap lengths ($o_h(b, e)$ and $o_v(b, e)$) can be formulated as maximum and minimum operations, smoothing techniques for HPWL can be directly used to smooth the net lengths and overlap lengths. Since the weighted-average wirelength model gives the best approximation to HPWL [19], we adopt the weighted-average wirelength model to smooth $L_{x,e}$, $L_{y,e}$, $o_h(b, e)$, and $o_v(b, e)$ in our analytical formulation. Furthermore, in our implementation, we set $\lambda_1 = \|\nabla \hat{C}(\mathbf{x}, \mathbf{y})\| / \|\nabla \hat{W}(\mathbf{x}, \mathbf{y})\|$, $\lambda_2 = \|\nabla \hat{C}(\mathbf{x}, \mathbf{y})\| / \|\nabla \hat{D}(\mathbf{x}, \mathbf{y})\|$, and $\lambda_3 = 1$. During the execution of our algorithm, λ_1 and λ_3 are fixed, and λ_2 is gradually doubled until blocks are spread sufficiently.

VI. ROUTABILITY-DRIVEN LEGALIZATION

In this section, we introduce the routability-driven legalization. Traditionally, legalization removes all overlaps of a given global placement result with minimal total displacement such that the objective which is optimized during global placement could be reserved. In order to minimize displacement, the movement of each cell is usually limited by a given search region. However, if cells are legalized into originally congested regions, it will make these regions more congested.

The basic idea of the routability-driven legalization is to consider routing congestion of candidate positions for legalizing cells. Given an initial global placement result, routing congested regions are first identified. We define the congestion cost of a given cell as the total number of routing edges connected to the cell. During the routability-driven legalization, we intend to legalize each cell with minimum displacement and minimum congestion cost.

VII. ROUTABILITY-DRIVEN DETAILED PLACEMENT

For existing detailed placement methods, cell matching and cell swapping are adopted in [22], [27], and [28] because of their great efficiency and effectiveness. However, [22] does not consider routability. Although [27] and [28] are routability-driven, they do not directly work on total overflow for cost evaluation, which may not capture routability well. Consequently, we revise these methods and

propose two routability-driven detailed placement techniques: 1) routability-driven cell matching and 2) routability-driven cell swapping.

A. Routability-Driven Cell Matching

In routability-driven cell matching, we find a minimum weighted matching of cells and placement locations within a selected window. The selected window is set to be 20 times the height of a single placement row and the maximum number of cells in a window is usually between 100 and 200 in our experiments. To evaluate the wiring cost correctly during matching, we only select independent cells, i.e., cells with no connections to other cells in the selected window. The matching cost is evaluated as a weighted sum of total overflow and wirelength

$$O(\mathbf{x}, \mathbf{y}) + \delta \times W(\mathbf{x}, \mathbf{y}) \quad (26)$$

where $O(\mathbf{x}, \mathbf{y})$ and $W(\mathbf{x}, \mathbf{y})$ are the respective total overflow and HPWL, and δ is a user-specified parameter for controlling the weights for simultaneous total overflow and wirelength optimization. To evaluate the total overflow, we first decompose all nets into two-pin nets and then perform L-shaped probabilistic routing. To save runtime, while computing each matching cost, we incrementally update the total overflow cost by ripping-up and rerouting the nets connected to the cell, instead of performing the probabilistic routing from scratch. With the weighted cost of total overflow and wirelength, we can optimize both costs simultaneously.

B. Routability-Driven Cell Swapping

Since cell matching can only handle independent cells, we apply routability-driven cell swapping to handle the remaining cells. Routability-driven cell swapping selects k adjacent cells in a single placement row and enumerates all possible permutations of these cells to find the minimum total overflow. k is usually small (which is 3 in our experiments) since we refine our circuit only locally during detailed placement. The cost function is the same as that defined in (26). We then apply the branch-and-bound technique to further speed up the swapping as in [22] and [29].

VIII. EXPERIMENTAL RESULTS

In this section, we first show the effectiveness of our proposed multilayer sigmoid potential model based on the ISPD'06 placement contest benchmarks [30]. Then, to evaluate our proposed routability-driven placement algorithm, we conducted several experiments based on the ICCAD'12 placement contest benchmarks [4]. We compared our algorithm with the top placers in the ICCAD'12 Routability-Driven Placement Contest based on the contest evaluation metrics. We implemented our algorithm in the C++ programming language. Our results were obtained on a Linux workstation with Intel Xeon E5-2620 2.0GHz CPU with 16 GB maximum allowed memory. The placement results in the contests [17] were derived from a Linux workstation with Intel Xeon X7560 2.27GHz CPU with 16 GB maximum allowed memory, which is faster than our environment. For fair comparisons, we scaled our machine performance to match with the contest environment.

TABLE I
RESULTING SCALED HPWLs (SHPWLs) WHICH INCLUDE SCALED OVERFLOW FACTOR WITH RESPECT TO THE GIVEN DENSITY TARGET, AND CPU TIME USING THE BELL-SHAPED POTENTIAL MODEL (BELL-SHAPED), THE SIGMOID POTENTIAL MODEL (SIGMOID), AND OUR PROPOSED MULTILAYER SIGMOID POTENTIAL MODEL (MULTISIGMOID) ON THE ISPD'06 PLACEMENT CONTEST BENCHMARKS [30]

Circuit Name	#Blocks	#Nets	Density Target	bell-shaped		sigmoid		multi-sigmoid	
				sHPWL ($\times 10^7$)	CPU (min)	sHPWL ($\times 10^7$)	CPU (min)	sHPWL ($\times 10^7$)	CPU (min)
adapte5	842k	867k	50	42.27	86.87	76.69	266.20	42.73	70.45
newblue1	330k	338k	80	6.09	15.22	7.25	28.25	6.09	9.02
newblue2	436k	465k	90	19.54	26.95	35.24	32.57	19.44	27.68
newblue3	482k	552k	80	27.74	29.53	27.42	20.27	28.06	31.50
newblue4	642k	637k	50	26.69	61.37	55.75	79.45	28.92	50.17
newblue5	1228k	1284k	50	53.00	171.55	113.47	305.82	52.57	98.67
newblue6	1248k	1288k	80	52.64	104.52	59.59	176.33	53.77	94.12
newblue7	2481k	2636k	80	109.60	227.13	138.12	334.32	111.74	155.80
Normalized				1.00	1.00	1.53	1.63	1.00	0.81

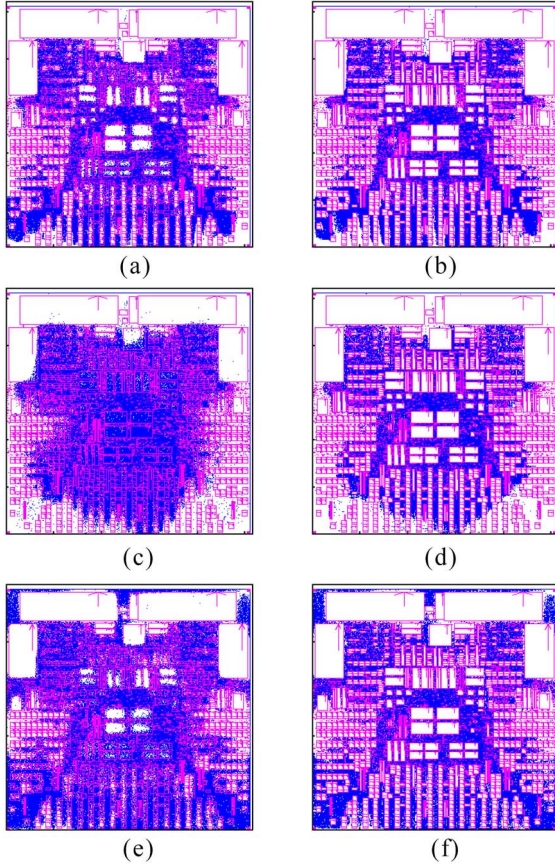


Fig. 9. Placement results of the circuit newblue5 generated using the bell-shaped potential model, the sigmoid potential model, and our proposed multilayer sigmoid potential model. Global placement result using (a) bell-shaped potential model, (c) sigmoid potential model, and (e) our proposed multilayer sigmoid potential model. Final placement result using (b) bell-shaped potential model, (d) sigmoid potential model, and (f) our proposed multilayer sigmoid potential model.

The ICCAD'12 contest [17] used three key metrics to evaluate the quality of the placement solutions [4]: 1) HPWL; 2) global-routing-based congestion; and 3) placement runtime, as follows:

$$W(\mathbf{x}, \mathbf{y}) \times (1 + \mu \times (RC(\mathbf{x}, \mathbf{y}) - 100)) \times (1 + t) \quad (27)$$

where $W(\mathbf{x}, \mathbf{y})$ is the HPWL, $RC(\mathbf{x}, \mathbf{y})$ is the global-routing-based congestion, μ is the penalty factor that scales the HPWL

TABLE II
STATISTICS OF THE ICCAD'12 PLACEMENT CONTEST BENCHMARKS [4]

Circuit Name	#Movable Blocks	#Nets	#Pins	#Fixed Macros	Util.%
superblue18	442k	469k	1864k	25k	67
superblue4	521k	568k	1884k	41k	70
superblue16	680k	697k	2281k	4k	69
superblue5	677k	787k	2500k	74k	77
superblue1	765k	823k	2861k	53k	69
superblue3	833k	898k	3111k	55k	73
superblue10	1046k	1159k	3894k	96k	70
superblue7	1271k	1340k	4935k	67k	76

to account for routing congestion, and t is the placement runtime factor (a $2\times$ speed-up or slow-down gets $\pm 4\%$ wirelength advantage). The global-routing-based congestion $RC(\mathbf{x}, \mathbf{y})$ is defined as

$$RC(\mathbf{x}, \mathbf{y}) = \max \left(100, \frac{\sum_{\epsilon=0.5, 1, 2, 5} (k_{\epsilon} \times ACE(\epsilon))}{\sum_{\epsilon=0.5, 1, 2, 5} k_{\epsilon}} \right) \quad (28)$$

where $ACE(\epsilon)$ is the average congestion of $\epsilon\%$ routing edges, and k_{ϵ} is the weight associated with $ACE(\epsilon)$. As in the contest [17], k_{ϵ} 's were set to 1.0, μ was set to 0.03, that is, for every 1% excessive routing congestion ($> 100\%$), there is a 3% wirelength penalty, and two routers, NCTU-GR 2.0 [31], and BFG-R 2.0 [32], were applied for evaluating global-routing-based congestion.

A. Density Model Comparisons

In this subsection, we validate our proposed multilayer sigmoid potential model based on the ISPD'06 placement contest benchmarks [30]. We integrated the bell-shaped potential model, the sigmoid potential model, and our multilayer sigmoid potential model into our placement algorithm without considering hierarchical designs and routability for fair comparison. For the multilayer sigmoid potential model, we set the bin size (number of bins) of dense bin grids the same as that for the bell-shaped and sigmoid potential models, i.e., $\sqrt{n}$, where n is the block number, and set the bin size of the loose bin grids to $(1/4)\sqrt{n}$.

Table I shows the experimental results. The scaled wirelength is calculated with $sHPWL = HPWL \times (1 + SOF)$, where SOF is the scaled overflow factor. We used the script provided by [33] to obtain the scaled overflow factor. As shown in the table, the average runtime of using the sigmoid model

TABLE III
RESULTING HPWLs AND CPU TIMES FOR PLACEMENT OF OUR ALGORITHM AND THE TOP PLACERS IN
THE ICCAD'12 DESIGN HIERARCHY AWARE ROUTABILITY-DRIVEN PLACEMENT CONTEST [17]
ON THE ICCAD'12 PLACEMENT CONTEST BENCHMARKS [4]

Circuit Name	SimPLR		Ripple		VDAPlace		NTUplace4h	
	HPWL ($\times 10^7$)	CPU (min)	HPWL ($\times 10^7$)	CPU (min)	HPWL ($\times 10^7$)	CPU (min)	HPWL ($\times 10^7$)	CPU (min)
superblue1	27.12	38.65	27.86	170.22	28.20	14.75	26.86	84.37
superblue3	33.83	45.10	33.31	251.90	41.26	19.55	31.76	92.22
superblue4	23.00	21.12	21.83	142.92	24.57	10.07	21.81	64.71
superblue5	34.89	35.90	34.47	180.55	35.78	14.57	34.92	86.37
superblue7	42.23	54.15	41.79	383.62	45.74	28.75	39.51	165.71
superblue10	60.77	80.62	57.80	438.53	58.87	21.93	58.55	153.14
superblue16	27.30	29.95	26.70	158.23	27.37	11.52	26.43	63.12
superblue18	16.41	27.42	16.37	183.15	19.16	10.97	15.08	54.75
Normalized	1.04	0.44	1.03	2.51	1.12	0.18	1.00	1.00

alone is much larger than that using the bell-shaped model. Furthermore, due to the insufficiency to spread blocks globally, using the sigmoid model alone usually obtains worse results than using the bell-shaped model. In contrast, using our proposed multilayer sigmoid potential model can achieve 19% shorter runtime with similar placement quality, compared with the bell-shaped model. Since the multilayer sigmoid potential model considers both global and local block densities, it helps spreading blocks into the placement region efficiently.

Fig. 9 shows the placement results of the circuit newblue5 generated using the bell-shaped potential model, the sigmoid potential model, and our multilayer sigmoid potential model. As shown in Fig. 9(a), (c), and (e), the sigmoid potential model alone cannot effectively spread blocks into the whole placement region. As a result, there are many overlaps between preplaced blocks and movable blocks, and thus the following legalization stage becomes more difficult [see Fig. 9(c) and (d)]. In contrast, using our multilayer sigmoid potential model can effectively facilitate the spreading process and achieve better placement results [see Fig. 9(e) and (f)].

B. ICCAD'12 Placement Contest Benchmarks

We also compared our algorithm with the top placers in the ICCAD'12 Design Hierarchy Aware Routability-Driven Placement Contest: SimPLR, Ripple, and VDAPlace. Table II lists the statistics of the ICCAD'12 placement contest benchmarks.

1) *HPWL and Runtime Comparisons:* Table III summarizes the comparisons of HPWL and placement runtime among our algorithm (NTUplace4h) and the top placers in the ICCAD'12 contest, where Columns 2–7 give the results of SimPLR, Ripple, and VDAPlace, respectively, and Columns 8 and 9 give the results of our algorithm. For fair comparisons, we scaled our machine performance to match with the contest environment. As shown in Table III, without considering the cost of global-routing-based congestion, our proposed algorithm can achieve 3% to 12% shorter HPWL compared with SimPLR, Ripple, and VDAPlace.

2) *Routing Congestion Comparisons:* In Tables IV and V, we compared the sHPWL which includes the global-routing-based congestion, and the total routed wirelength evaluated by NCTU-GR 2.0 [31] and BFG-R 2.0 [32], respectively.

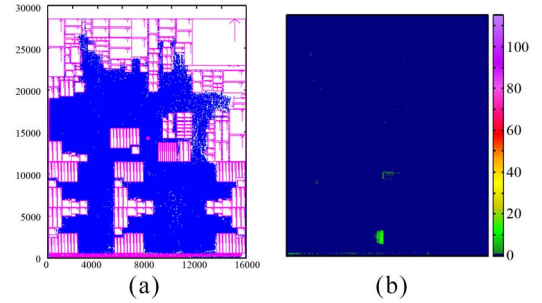


Fig. 10. (a) Placement result and (b) routing congestion map routed by NCTU-GR 2.0 for superblue7. Our algorithm achieves 9%, 8%, and 26% shorter sHPWL compared with SimPLR, Ripple, and VDAPlace, respectively.

As shown in Tables IV and V, our algorithm can achieve the best global-routing-based congestion and sHPWL for both routers. The results show the robustness of our proposed algorithm. Fig. 10 shows the placement result and routing congestion map for superblue7.

3) *ICCAD'12 Placement Contest Overall Score Comparisons:* Table VI summarizes the overall evaluation, where normalized costs which include HPWL, global-routing-based congestion, and placement runtime are compared. Columns 2–7 give the results of Ripple, SimPLR, and VDAPlace, and Columns 8 and 9 give the results of our algorithm. Based on NCTU-GR 2.0 and BFG-R 2.0, our algorithm can achieve the best overall score, with 2%, 10%, and 29% smaller normalized cost compared to SimPLR, Ripple, and VDAPlace, respectively.

4) *Individual Technique Comparisons:* To show the effectiveness and efficiency of our proposed techniques, we compared our proposed algorithm without and with each technique. Fig. 11 summarizes the experimental results. As shown in the figure, all our proposed techniques can achieve at least 2% better sHPWL with 11% runtime penalty to even 6% runtime gain; some techniques can even improve the placement runtime. For example, the design hierarchy identification can guide the placement for better wirelength and routability, and the narrow channel handling can prevent blocks from getting stuck in the narrow channels. Both techniques can improve the convergency of the global placement. Although using pin density and overflow minimization might require additional computational runtime, both techniques

TABLE IV
RESULTING GLOBAL-ROUTING-BASED CONGESTIONS (RC) EVALUATED BY NCTU-GR 2.0 [31] AND sHPWLs OF OUR ALGORITHM AND THE TOP PLACERS IN THE ICCAD'12 DESIGN HIERARCHY AWARE ROUTABILITY-DRIVEN PLACEMENT CONTEST [17] ON THE ICCAD'12 PLACEMENT CONTEST BENCHMARKS [4]

Circuit Name	SimPLR		Ripple		VDAPlace		NTUplace4h	
	RC	sHPWL ($\times 10^4$)	RC	sHPWL ($\times 10^4$)	RC	sHPWL ($\times 10^4$)	RC	sHPWL ($\times 10^4$)
superblue1	100.94	27.89	101.23	28.89	110.39	36.98	101.58	28.13
superblue3	100.56	34.39	102.73	36.04	117.69	63.16	102.98	34.59
superblue4	101.94	24.34	101.32	22.69	104.96	28.23	101.90	23.05
superblue5	101.09	36.03	100.38	34.86	119.72	56.96	100.61	35.56
superblue7	100.71	43.13	100.89	42.91	103.11	50.01	100.26	39.82
superblue10	104.56	69.09	101.91	61.11	125.33	103.60	101.77	61.67
superblue16	101.55	28.57	102.13	28.40	105.91	32.23	101.91	27.94
superblue18	103.72	18.23	103.15	17.91	117.31	29.11	102.84	16.36
Normalized	1.00	1.05	1.00	1.03	1.11	1.48	1.00	1.00

TABLE V
RESULTING GLOBAL-ROUTING-BASED CONGESTIONS (RC) EVALUATED BY BFG-R 2.0 [32] AND sHPWLs OF OUR ALGORITHM AND THE TOP PLACERS IN THE ICCAD'12 DESIGN HIERARCHY AWARE ROUTABILITY-DRIVEN PLACEMENT CONTEST [17] ON THE ICCAD'12 PLACEMENT CONTEST BENCHMARKS [4]

Circuit Name	SimPLR		Ripple		VDAPlace		NTUplace4h	
	RC	sHPWL ($\times 10^4$)	RC	sHPWL ($\times 10^4$)	RC	sHPWL ($\times 10^4$)	RC	sHPWL ($\times 10^4$)
superblue1	103.89	30.28	106.64	33.41	109.27	36.04	103.61	29.77
superblue3	104.74	38.64	105.75	39.06	114.55	59.28	105.60	37.09
superblue4	108.05	28.56	107.27	26.59	103.88	27.43	107.75	26.88
superblue5	104.20	39.29	102.00	36.54	107.31	43.63	103.28	38.35
superblue7	102.39	45.25	102.57	45.02	105.37	53.10	101.13	40.84
superblue10	110.87	80.58	107.50	70.80	123.44	100.26	108.03	72.65
superblue16	104.72	31.17	103.23	29.29	110.87	36.30	105.16	30.51
superblue18	116.66	24.61	111.62	22.07	131.16	37.07	104.32	17.03
Normalized	1.02	1.10	1.01	1.06	1.08	1.38	1.00	1.00

TABLE VI
RESULTING NORMALIZED COSTS WHICH INCLUDE HPWL, GLOBAL-ROUTING-BASED CONGESTION AND CPU TIMES FOR PLACEMENT OF OUR ALGORITHM AND THE TOP PLACERS IN THE ICCAD'12 DESIGN HIERARCHY AWARE ROUTABILITY-DRIVEN PLACEMENT CONTEST [17] ON THE ICCAD'12 PLACEMENT CONTEST BENCHMARKS [4]. TWO ROUTERS, NCTU-GR 2.0 [31] AND BFG-R 2.0 [32], ARE APPLIED SEPARATELY FOR EVALUATING GLOBAL-ROUTING-BASED CONGESTIONS

Circuit Name	SimPLR		Ripple		VDAPlace		NTUplace4h	
	NCTU-GR	BFG-R	NCTU-GR	BFG-R	NCTU-GR	BFG-R	NCTU-GR	BFG-R
superblue1	0.95	0.97	1.07	1.17	1.18	1.09	1.00	1.00
superblue3	0.95	1.00	1.10	1.11	1.66	1.45	1.00	1.00
superblue4	0.99	0.99	1.03	1.03	1.09	0.91	1.00	1.00
superblue5	0.96	0.97	1.02	0.99	1.44	1.02	1.00	1.00
superblue7	1.01	1.04	1.13	1.16	1.13	1.17	1.00	1.00
superblue10	1.08	1.07	1.05	1.03	1.49	1.22	1.00	1.00
superblue16	0.98	0.98	1.07	1.01	1.04	1.07	1.00	1.00
superblue18	1.07	1.39	1.17	1.39	1.61	1.97	1.00	1.00
Average	1.02		1.10		1.29		1.00	

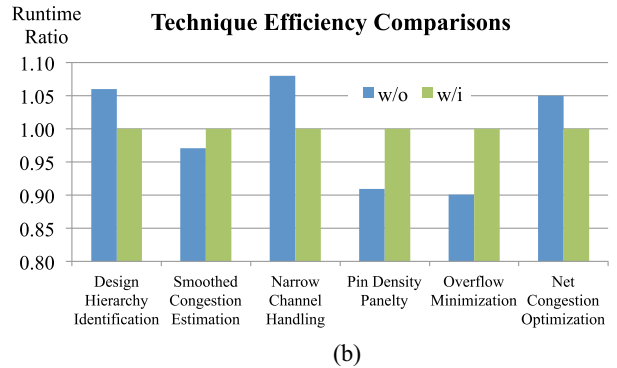
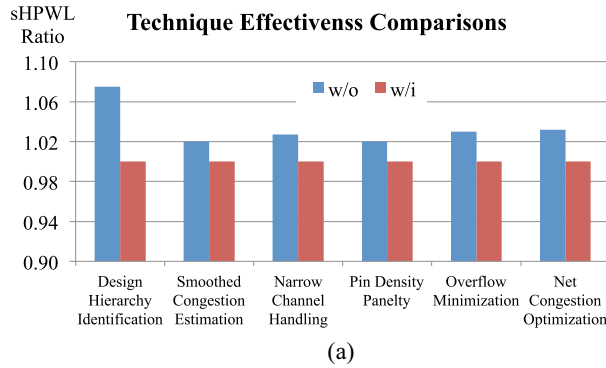


Fig. 11. (a) Effectiveness and (b) efficiency comparisons of individual techniques proposed in this paper.

can improve the sHPWL effectively. Among these proposed techniques, the design hierarchy identification can achieve the largest improvement on both sHPWL and placement

runtime. We show the details of the effectiveness of the design hierarchy identification technique in the following section.

TABLE VII
RESULTING HPWLs, CPU TIMES, AND SHPWLS EVALUATED BY NCTU-GR 2.0 [31] AND BFG-R 2.0 [32] OF OUR
PROPOSED ALGORITHM WITH AND WITHOUT THE PROPOSED DESIGN HIERARCHY IDENTIFICATION
TECHNIQUE ON THE ICCAD'12 PLACEMENT CONTEST BENCHMARKS [4]

Circuit Name	NTUplace4h w/o Hierarchy Identification				NTUplace4h w/i Hierarchy Identification			
	HPWL ($\times 10^7$)	CPU (min)	sHPWL (NCTU-GR) ($\times 10^7$)	sHPWL (BFG-R) ($\times 10^7$)	HPWL ($\times 10^7$)	CPU (min)	sHPWL (NCTU-GR) ($\times 10^7$)	sHPWL (BFG-R) ($\times 10^7$)
superblue1	27.01	90.40	28.47	31.26	26.86	84.37	28.13	29.77
superblue3	32.21	106.04	41.86	41.38	31.76	92.22	34.59	37.09
superblue4	21.59	69.10	23.99	27.71	21.81	64.71	23.05	26.88
superblue5	34.73	83.94	37.94	40.57	34.92	86.37	35.56	38.35
superblue7	40.04	169.02	40.62	42.34	39.51	165.71	39.82	40.84
superblue10	57.07	138.39	67.52	77.37	58.55	153.14	61.67	72.65
superblue16	26.65	75.66	28.64	31.43	26.43	63.12	27.94	30.52
superblue18	16.01	58.49	18.47	19.43	15.08	54.75	16.36	17.03
Normalized	1.01	1.06	1.07	1.07	1.00	1.00	1.00	1.00

5) Design Hierarchy Identification Effectiveness Validation:

As mentioned in Section I, hierarchical design methodologies are often employed in modern circuit designs for faster turnaround time. In this section, we validate the effectiveness of our proposed design hierarchy identification technique for design hierarchy identification. Table VII summarizes the comparison results between our proposed algorithm, without and with hierarchy identification. As shown in the table, the proposed hierarchy identification technique can achieve averagely 7% better sHPWL. As mentioned in Section IV, the proposed design hierarchy identification technique can help preserve design hierarchies during the uncoarsening stage of global placement and guide the placement for better wirelength and routability. Moreover, it can also improve the convergency of the global placement, as shown in Table VII, the placement runtime can averagely be reduced by 6% with hierarchy identification.

IX. CONCLUSION

In this paper, we have presented a novel routability-driven analytical placement framework for hierarchical mixed-size circuit designs. Similar to most existing works, our algorithm consists of three main stages, global placement, legalization, and detailed placement. Our algorithm first adopts a novel design hierarchy identification technique to effectively identify design hierarchies and guide placement for better wirelength and routability. During global placement, our algorithm then optimizes routability for global routing from four aspects: 1) narrow channel handling; 2) pin density; 3) routing overflow optimization; and 4) net congestion optimization. Unlike most existing works which usually optimize routability by implicit or explicit reallocating whitespace to improve routability, our algorithm optimizes routing overflow by a novel sigmoid function during global placement and adopts a net-congestion-optimization technique to optimize wirelength and routability during placement. Techniques for legalization and detailed placement are further proposed to improve circuit routability. Experimental results have shown that our algorithm can achieve the best quality (both the average overflow and wirelength), compared with the participating teams for the 2012 ICCAD Design Hierarchy Aware Routability-driven Placement Contest [17].

REFERENCES

- [1] M.-K. Hsu, Y.-F. Chen, C.-C. Huang, and Y.-W. Chang, "Routability-driven placement for hierarchical mixed-size circuit designs," in *Proc. 50th ACM/IEEE Des. Autom. Conf.*, Austin, TX, USA, 2013, pp. 1–6.
- [2] M.-K. Hsu, S. Chou, T.-H. Lin, and Y.-W. Chang, "Routability-driven analytical placement for mixed-size circuit designs," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, San Jose, CA, USA, 2011, pp. 80–84.
- [3] Y.-L. Chuang *et al.*, "Design-hierarchy aware mixed-size placement for routability optimization," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, San Jose, CA, USA, 2010, pp. 663–668.
- [4] N. Viswanathan, C. Alpert, C. Sze, Z. Li, and Y. Wei, "ICCAD-2012 CAD contest in design hierarchy aware routability-driven placement and benchmark suite," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, San Jose, CA, USA, 2012, pp. 345–348.
- [5] X. Yang, B.-K. Choi, and M. Sarrafzadeh, "Routability-driven white space allocation for fixed-die standard-cell placement," in *Proc. ACM Int. Symp. Phys. Des.*, 2002, pp. 410–419.
- [6] C. Li, M. Xie, C.-K. Koh, J. Cong, and P. H. Madden, "Routability-driven placement and white space allocation," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2004, pp. 394–401.
- [7] J. A. Roy, J. F. Lu, and I. L. Markov, "Seeing the forest and the trees: Steiner wirelength optimization in placement," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 26, no. 4, pp. 632–644, Apr. 2007.
- [8] U. Brenner, M. Struzyna, and J. Vygen, "BonnPlace: Placement of leading-edge chips by advanced combinatorial algorithms," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 9, pp. 1607–1620, Sep. 2008.
- [9] J. A. Roy, N. Viswanathan, G.-J. Nam, C. J. Alpert, and I. L. Markov, "CRISP: Congestion reduction by iterated spreading during placement," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, San Jose, CA, USA, 2009, pp. 357–362.
- [10] X. He *et al.*, "Ripple2.0: High quality routability-driven placement via global router integration," in *Proc. ACM/IEEE Des. Autom. Conf.*, Austin, TX, USA, 2013, pp. 1–6.
- [11] X. He *et al.*, "Ripple: An effective routability-driven placer by iterative cell movement," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, San Jose, CA, USA, 2011, pp. 74–79.
- [12] J. Hu, M.-C. Kim, and I. L. Markov, "Taming the complexity of coordinated place and route," in *Proc. ACM/IEEE Des. Autom. Conf.*, Austin, TX, USA, 2013, pp. 1–7.
- [13] M.-C. Kim, J. Hu, D.-J. Lee, and I. L. Markov, "A SimPLR method for routability-driven placement," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, San Jose, CA, USA, 2011, pp. 67–73.
- [14] P. Spindler and F. M. Johannes, "Fast and accurate routing demand estimation for efficient routability-driven placement," in *Proc. ACM/IEEE Des. Autom. Test Eur.*, Nice, France, 2007, pp. 1226–1231.
- [15] Z.-W. Jiang, B.-Y. Su, and Y.-W. Chang, "Routability-driven analytical placement by net overlapping removal for large-scale mixed-size designs," in *Proc. ACM/IEEE Des. Autom. Conf.*, Anaheim, CA, USA, 2008, pp. 167–172.
- [16] K. Tsota, C. Koh, and V. Balakrishnan, "Guiding global placement with wire density," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2008, pp. 212–217.

- [17] (2012, Nov. 13). *ICCAD 2012 Placement Contest* [Online]. Available: http://cad_contest.cs.nctu.edu.tw/CAD-contest-at-ICCAD2012/
- [18] Y.-W. Chang, Z.-W. Jiang, and T.-C. Chen, "Essential issues in analytical placement algorithms," *IPSP Trans. Syst. LSI Des. Methodol.*, vol. 2, pp. 145–166, Aug. 2009.
- [19] M.-K. Hsu, V. Balabanov, and Y.-W. Chang, "TSV-aware analytical placement for 3-D IC designs based on a novel weighted-average wire-length model," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 32, no. 4, pp. 497–509, Apr. 2013.
- [20] S. Chou, M.-K. Hsu, and Y.-W. Chang, "Structure-aware placement for datapath-intensive circuit designs," in *Proc. ACM/IEEE Des. Autom. Conf.*, 2012, pp. 762–767.
- [21] T. F. Chan, J. Cong, T. Kong, and J. R. Shinnerl, "Multilevel optimization for large-scale circuit placement," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, San Jose, CA, USA, 2000, pp. 171–176.
- [22] T.-C. Chen, Z.-W. Jiang, T.-C. Hsu, H.-C. Chen, and Y.-W. Chang, "NTUplace3: An analytical placer for large-scale mixed-size designs with preplaced blocks and density constraints," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 7, pp. 1228–1240, Jul. 2008.
- [23] J. M. Kleinhans, G. Sigl, F. M. Johannes, and K. J. Antreich, "GORDIAN: VLSI placement by quadratic programming and slicing optimization," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 10, no. 3, pp. 356–365, Mar. 1991.
- [24] A. B. Kahng and Q. Wang, "An analytic placer for mixed-size placement and timing-driven placement," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2004, pp. 565–572.
- [25] R. C. Gonzales and R. E. Woods, *Digital Image Processing*. Reading, MA, USA: Addison-Wesley, 1992.
- [26] C. Chu and Y.-C. Wong, "FLUTE: Fast lookup table based rectilinear steiner minimal tree algorithm for VLSI design," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 1, pp. 70–83, Jan. 2008.
- [27] K.-R. Dai, C.-H. Lu, and Y.-L. Li, "GRPlacer: Improving routability and wire-length of global routing with circuit replacement," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, San Jose, CA, USA, 2009, pp. 351–356.
- [28] Y. Zhang and C. Chu, "CROP: Fast and effective congestion refinement of placement," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2009, pp. 344–350.
- [29] Z.-W. Jiang, T.-C. Chen, T.-C. Hsu, H.-C. Chen, and Y.-W. Chang, "NTUplace2: A hybrid placer using partitioning and analytical techniques," in *Proc. ACM Int. Symp. Phys. Des.*, 2006, pp. 215–217.
- [30] G.-J. Nam, "ISPD 2006 placement contest: Benchmarks suite and results," in *Proc. ACM Int. Symp. Phys. Des.*, 2006, p. 167.
- [31] W.-H. Liu, W.-C. Kao, Y.-L. Li, and K.-Y. Chao, "Multi-threaded collision-aware global routing with bounded-length maze routing," in *Proc. ACM/IEEE Des. Autom. Conf.*, Anaheim, CA, USA, 2010, pp. 200–205.
- [32] J. Hu, J. A. Roy, and I. L. Markov, "Completing high-quality global routes," in *Proc. ACM Int. Symp. Phys. Des.*, 2010, pp. 35–41.
- [33] (2006, Apr. 3). *ISPD 2006 Placement Contest* [Online]. Available: <http://www.sigda.org/ispd2006/contest.html>



Meng-Kai Hsu received the B.S. degrees in electronics engineering and computer science from the National Chiao Tung University, Hsinchu, Taiwan, and the Ph.D. degree in electronics engineering from the National Taiwan University, Taipei, Taiwan, in 2007 and 2012, respectively.

His current research interests include VLSI physical design, floorplanning, and mixed-size placement.

Mr. Hsu is an Honorary Member of the Phi Tau Phi Scholastic Honor Society of Taiwan.



Yi-Fang Chen received the B.S. degree from the National Chiao Tung University, Hsinchu, Taiwan, and the M.S. degree in electronics engineering from the National Taiwan University, Taipei, Taiwan, in 2011 and 2013, respectively, both in electronics engineering.

His current research interests include logic synthesis, VLSI physical design, and placement.



Chau-Chin Huang received the B.S. degree in electrical engineering from the National Cheng Kung University, Tainan, Taiwan, in 2012. He is currently pursuing the Ph.D. degree from the Graduate Institute of Electronics Engineering, the National Taiwan University, Taipei, Taiwan.

His current research interests include placement and datapath extraction.



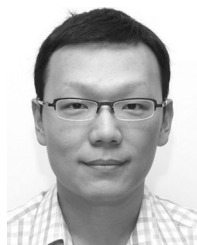
Sheng Chou received the B.S. degree in computer science and information engineering from the National Cheng Kung University, Tainan, Taiwan, and the M.S. degree in electronics engineering from the National Taiwan University, Taipei, Taiwan, in 2010 and 2012, respectively.

His current research interests include very large scale integration physical design, clock network synthesis, and analytical placement.



Tzu-Hen Lin received the B.S. degree from the National Chiao Tung University, Hsinchu, Taiwan, and the M.S. degree from the National Taiwan University, Taipei, Taiwan, in 2010 and 2012, respectively, both in electronics engineering.

His current research interests include VLSI physical design and FPGA placement.



Tung-Chieh Chen received the B.S. and Ph.D. degrees in electrical engineering from the National Taiwan University, Taipei, Taiwan, in 2003 and 2008, respectively.

He is currently a Research and Development Manager with Design Group, Synopsys, Tainan, Taiwan. He was a Visiting Scholar with the University of Texas at Austin, Austin, TX, USA, in 2007. He joined SpringSoft, Hsinchu, Taiwan, in 2008 and then entered Synopsys, Mountain View, CA, USA, in 2012, by acquisition. His current

research interests include very large scale integration physical design, custom layout design automation, floorplanning, and placement.



Yao-Wen Chang (S'94–A'96–M'96–SM'12–F'13) received the B.S. degree from the National Taiwan University (NTU), Taipei, Taiwan, and the M.S. and Ph.D. degrees from the University of Texas at Austin, Austin, TX, USA, in 1988, 1993, and 1996, respectively, all in computer science.

He is currently an Associate Dean with the College of Electrical Engineering and Computer Science, and a Distinguished Professor with the Department of Electrical Engineering, NTU. He has co-edited one textbook on electronic design automation and co-authored one book on routing and over 240 ACM/IEEE conference and journal papers in these areas. His current research interests include physical design and manufacturability for integrated circuits.